

Fiche 300 — Prendre R en main : environnement, session, espace de travail

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> — Notes on R: A Programming Environment for Data Analysis and Graphics, version 4.6.1 (2026-06-24) — préface, chapitre 1 « Introduction and preliminaries », annexe A « A sample session », annexe B « Invoking R », annexe C « The command-line editor »
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §10.3.2 « Identifiers », §10.3.3 « Reserved words », §10.3.5 « Separators »
Difficulté	Fondamental — la première heure de R, celle qui conditionne toutes les autres
Temps d'étude estimé	1 h 15
Prérequis	Aucun. Savoir ouvrir un terminal aide pour les parties « ligne de commande » et « scripts ».
Concepts clés	environnement contre logiciel de statistique, objet de résultat, session, <code>q()</code> , espace de travail, <code>.RData</code> , <code>.Rhistory</code> , <code>ls()</code> , <code>rm()</code> , expression contre assignation, <code><-</code> , sensibilité à la casse, mots réservés, <code>help()</code> , <code>??</code> , <code>example()</code> , <code>source()</code> , <code>sink()</code> , séquence de démarrage, <code>.Rprofile</code> , <code>.First()</code> , <code>--vanilla</code> , <code>R CMD</code> , <code>Rscript</code> , <code>commandArgs()</code>
À retenir en priorité	La distinction expression / assignation · le cycle de vie de l'espace de travail · les quatre formes de l'aide · <code>--vanilla</code> et la séquence de démarrage · <code>Rscript</code> + <code>commandArgs(TRUE)</code> .

Vue d'ensemble

CE QU'EST R	un ENVIRONNEMENT integre (donnees, calcul, graphiques, langage S) pas un logiciel qui « sort un listing » : il STOCKE le resultat
LA BOUCLE	\$ cd work ; R -> > commandes -> > q() -> sauver ? oui/non
ESPACE DE TRAVAIL	objets nommes ; ls() les liste ; rm() les efface a la sortie -> .RData (objets) + .Rhistory (commandes) au demarrage suivant DANS LE MEME DOSSIER -> tout revient

DEUX COMMANDES	expression	1/x	evaluee, AFFICHEE, valeur PERDUE
	assignation	x <- 1/y	evaluee, RANGEe, rien d'affiche
L'AIDE	?solve	help("[[")	??solve example(solve)
	^ nom	^ caracteres	^ recherche ^ faire tourner
DEMARRAGE	Renviron -> Rprofile.site -> .Rprofile -> .RData -> .First() --vanilla coupe TOUT cela		
HORS SESSION	source("f.R")	R CMD BATCH f.R	Rscript f.R arg1 arg2

Le problème posé. *R* est « *an integrated suite of software facilities for data manipulation, calculation and graphical display* » — une **suite intégrée**, et non une collection d'outils juxtaposés. Toute la difficulté du débutant vient de là : il cherche un logiciel qui **produit un rapport**, alors qu'il a devant lui un **langage qui produit des objets**.

INTUITION — LA PHRASE QUI EXPLIQUE TOUT LE RESTE.

« *There is an important difference in philosophy between S (and hence R) and the other main statistical systems. In S a statistical analysis is normally done as a series of steps, with intermediate results being stored in objects. Thus whereas SAS and SPSS will give copious output from a regression or discriminant analysis, R will give minimal output and store the results in a fit object for subsequent interrogation by further R functions.* » Autrement dit : **R vous rend peu de texte et beaucoup d'objet**. Ce qui ressemble à de l'avarice est en réalité de la générosité — le résultat reste **interrogeable**.

● Concept 1 — Ce qu'est R : un environnement, pas un outil

DÉFINITION (§1.1).

R est une **suite intégrée** de moyens logiciels pour la **manipulation de données**, le **calcul** et l'**affichage graphique**.

Le cours en donne cinq composantes, à connaître comme telles :

#	Composante (§1.1)	Ce que cela veut dire concrètement
1	<i>an effective data handling and storage facility</i>	les données vivent dans la session , sous forme d'objets nommés
2	<i>a suite of operators for calculations on arrays, in particular matrices</i>	l'algèbre linéaire est native , pas une extension
3	<i>a large, coherent, integrated collection of intermediate tools for data analysis</i>	des outils intermédiaires : ils se composent entre eux
4	<i>graphical facilities for data analysis and display</i>	à l'écran ou sur fichier (<i>hardcopy</i>)
5	<i>a well developed, simple and effective programming language (called « S »)</i>	conditionnelles, boucles, fonctions récurives définies par l'utilisateur, entrées-sorties

EN CLAIR — POURQUOI LE MOT « ENVIRONNEMENT » (§1.1).

Le terme « environment » vise à le caractériser comme **un système entièrement planifié et cohérent**, plutôt que comme **une accréation incrémentale d'outils très spécifiques et inflexibles**, comme c'est fréquemment le cas dans d'autres logiciels d'analyse de données. La conséquence pratique est immédiate : **ce que vous apprenez sur un objet vaut pour tous les autres**.

`length()`, `names()`, `[`, `summary()` ne sont pas des commandes propres à un type de données — ce sont des mécanismes généraux.

La cinquième composante est celle qui vous intéresse. « *Indeed most of the system supplied functions are themselves written in the S language.* » Les fonctions de R sont écrites **en R**. Cela signifie qu'il n'y a **aucune frontière** entre « utiliser R » et « programmer en R » : taper le nom d'une fonction sans parenthèses affiche son code source, et ce code est du R ordinaire que vous pouvez lire, copier, modifier.

1.1 D'où vient R : la filiation S

NOTATION ET VOCABULAIRE (§1.2).

R peut être vu comme **une implémentation du langage S**, développé aux **Bell Laboratories** par **Rick Becker, John Chambers et Allan Wilks**, et qui sert aussi de base aux systèmes **S-Plus**.

L'évolution du langage S est caractérisée par quatre livres de John Chambers et de ses coauteurs. Trois comptent, et le cours les nomme explicitement :

Ouvrage	Auteurs	Ce qu'il fonde	Surnom (Références, p. 92)
<i>The New S Language: A Programming Environment for Data Analysis and Graphics</i> (1988)	Becker, Chambers & Wilks	la référence de base pour R	le « Blue Book »
<i>Statistical Models in S</i> (1992)	Chambers & Hastie (éd.)	les nouveautés de la version 1991 : les modèles statistiques (voir fiche 312)	le « White Book »
<i>Programming with Data</i> (1998)	Chambers	les méthodes et classes formelles du paquet <code>methods</code> (S4)	le « Green Book »

La documentation de S/S-Plus peut généralement être utilisée avec R, **en gardant à l'esprit les différences entre les implémentations de S**. Ce n'est donc pas une équivalence, mais une proximité.

1.2 R et la statistique — une nuance importante

CE QUE LE COURS PREND SOIN DE DIRE (§1.3).

Notre présentation de l'environnement R **n'a pas mentionné la statistique**, alors que beaucoup de gens utilisent R **comme un système de statistique**. Nous préférons y voir **un environnement dans lequel de nombreuses techniques statistiques classiques et modernes ont été implémentées**.

- **Quelques-unes** de ces techniques sont **dans R lui-même** (la base).
- **Beaucoup** sont fournies **sous forme de paquets** (*packages*).
- Environ **25 paquets** sont livrés avec R — les paquets dits « **standard** » et « **recommended** ».
- Beaucoup d'autres viennent de la famille de sites **CRAN** (<https://CRAN.R-project.org>) et d'ailleurs. Voir fiche 314.

« *Most classical statistics and much of the latest methodology is available for use with R, **but users may need to be prepared to do a little work to find it.*** » Le cours vous prévient : **la difficulté n'est pas la disponibilité, c'est la découvrabilité**. D'où l'importance disproportionnée de la section « aide » plus bas.

REMARQUE — LA DURÉE DE VIE DU CODE R (§1.1).

« *However, most programs written in R are essentially **ephemeral**, written for a single piece of data analysis.* » C'est une observation du cours, pas un jugement : **la plupart du code R est jetable**. Elle a une conséquence méthodologique — le jour où un script cesse d'être jetable, il faut le transformer en **fonctions** (fiche 308), puis en **paquet** (fiche 318).

● Concept 2 — La session : ouvrir, travailler, fermer

DÉFINITION.

Une **session R** est l'intervalle entre le lancement du programme et l'appel de `q()`. Tout ce que vous créez entre les deux vit dans une zone de mémoire nommée **l'espace de travail** (*workspace*).

L'invite (§1.5). Quand vous utilisez le programme R, il émet une **invite** (prompt) lorsqu'il attend des commandes. L'invite par défaut est `>`, qui sous UNIX peut être la même que celle du shell, si bien qu'on peut avoir l'impression **qu'il ne se passe rien**. Le cours convient que l'invite du shell UNIX est notée `$`.

2.1 La procédure recommandée, pas à pas

Le cours donne une procédure en quatre étapes pour la **première** utilisation (§1.5). Elle paraît triviale : elle ne l'est pas, parce que **l'étape 1 détermine tout le comportement de sauvegarde** décrit au concept 3.

Étape 1 — créer un sous-dossier dédié. Il deviendra le **répertoire de travail** pour ce problème-là, et **pour lui seul**.

```
$ mkdir work
$ cd work
```

Étape 2 — lancer R depuis ce dossier.

\$ R

Étape 3 — travailler. R émet `>`, vous tapez des commandes.

Étape 4 — quitter.

`q()`

« *At this point you will be asked whether you want to save the data from your R session.* » Sur certains systèmes une **boîte de dialogue** apparaît, sur d'autres une invite textuelle à laquelle on répond `yes`, `no` ou `cancel` — une abréviation d'une seule lettre suffit — pour respectivement **sauvegarder puis quitter**, **quitter sans sauvegarder**, ou **revenir dans la session**.

`cancel` n'est pas « annuler la sauvegarde » : c'est « *return to the R session* ». Vous ne quittez pas. C'est le premier réflexe faux du débutant.

Les sessions suivantes sont simples (§1.5) : `cd work`, `R`, travailler, `q()`.

Sous Windows (§1.5). La procédure à suivre est fondamentalement la même. Créer un dossier comme répertoire de travail, et le renseigner dans le champ **Start In** de votre raccourci R. Puis lancer R en double-cliquant sur l'icône.

Le piège du raccourci Windows. Le champ *Start In* est l'étape 1 de la procédure UNIX. Si vous l'ignorez, tous vos projets partagent le même répertoire de travail, donc **le même .RData** — et le concept 3 devient un piège au lieu d'un confort.

● Concept 3 — L'espace de travail : ce qui survit, ce qui disparaît

DÉFINITION (§1.11).

Les entités que R crée et manipule sont appelées des **objets** (objects). Ce peuvent être des variables, des tableaux de nombres, des chaînes de caractères, des fonctions, ou des structures plus générales construites à partir de ces composants.

La collection d'objets actuellement stockés s'appelle **l'espace de travail** (workspace).

3.1 Voir et effacer

```
objects()      # les noms de (la plupart des) objets stockés
ls()           # strictement équivalent
rm(x, y, z, ink, junk, temp, foo, bar)  # effacement
```

Le « la plupart » n'est pas une coquetterie. Le cours écrit « *(most of) the objects* ». La raison est donnée par la *R Language Definition* (§10.3.2) : les identifiants **commençant par un point** ne sont pas listés par défaut par `ls`. C'est ainsi que `.RData` peut contenir un `.First` que `ls()` ne montre pas.

ENRICHISSEMENT PÉDAGOGIQUE (HORS COURS) — VOIR AUSSI LES OBJETS CACHÉS.

`ls(all.names = TRUE)` lève ce filtre. Le manuel *An Introduction to R* ne le mentionne pas ici ; l'argument est documenté dans la page d'aide de `ls` (*Reference Index*, paquet `base`).

3.2 Le cycle de sauvegarde — le mécanisme complet

Le mécanisme (§1.11). Tous les objets créés pendant une session R peuvent être stockés **de façon permanente** dans un fichier, pour un usage dans de futures sessions. À la fin de chaque session R, on vous donne la possibilité de sauvegarder tous les objets disponibles.

Ce qui est écrit	Contenu	Quand
<code>.RData</code>	les objets de l'espace de travail	à la sortie, si vous répondez <code>yes</code>
<code>.Rhistory</code>	les lignes de commande utilisées dans la session	à la sortie, en même temps

« When R is started at later time **from the same directory** it **reloads the workspace from this file**. At the same time the associated commands history is reloaded. »

Les trois conditions pour que la reprise fonctionne, dans l'ordre où elles échouent en pratique :

1. vous avez répondu `yes` à la question de sortie ;
2. vous relancez R **depuis le même répertoire** ;
3. R n'a pas été lancé avec une option qui coupe la restauration (`--no-restore` , `--vanilla` — concept 8).

« The leading "dot" in this file name makes it **invisible** in normal file listings in UNIX, and in default GUI file listings on macOS and Windows. » (note 5, §1.11) Les deux fichiers **existent et ne se voient pas**. Un débutant qui « ne retrouve plus ses données » les a presque toujours sous les yeux.

3.3 Pourquoi un dossier par analyse — l'argument du cours

MÉTHODE — LA RECOMMANDATION, ET SA JUSTIFICATION (§1.11).

« It is recommended that you should use **separate working directories** for analyses conducted with R. It is quite common for objects with names `x` and `y` to be created during an analysis. Names like this are often meaningful in the context of **a single analysis**, but it can be quite hard to decide what they might be when **several analyses have been conducted in the same directory**. »

L'argument n'est pas l'hygiène : c'est que `x` **est un nom parfaitement bon tant qu'il n'y en a qu'un**. La séparation des dossiers est ce qui rend les **noms courts** utilisables — et donc le travail interactif agréable.

Énoncé. Dans `~/work` , une session crée `x` , `y` et `fm` . L'utilisateur quitte avec `q()` et répond `y` . Le lendemain il lance R depuis son dossier personnel `~` , tape `ls()` et ne voit rien. Que s'est-il passé, et que doit-il faire ?

Étape 1 — *ce qui a été écrit*. La réponse `y` a produit `~/work/.RData` (contenant `x`, `y`, `fm`) et `~/work/.Rhistory`. L'écriture a bien eu lieu.

Étape 2 — *ce qui est relu*. R relit `.RData` **du répertoire courant**. Lancé depuis `~`, il cherche `~/ .RData` — un autre fichier, probablement inexistant. Il démarre donc sur un espace de travail vide, et `ls()` renvoie `character(0)`.

Étape 3 — *pourquoi c'est invisible*. Aucune erreur n'est levée : l'absence de `.RData` est le cas **normal** d'une première session dans un dossier. Le silence de R est ici légitime, et c'est ce qui rend la situation déroutante.

Étape 4 — *le remède immédiat*. Quitter, retourner dans le bon dossier, relancer :

```
$ cd ~/work
$ R
```

Étape 5 — *vérifier*. `ls()` doit maintenant renvoyer `"fm" "x" "y"`, et les flèches du clavier doivent faire remonter l'historique de la veille (§1.9).

Étape 6 — *la leçon*. Le répertoire de lancement **est** l'identité du projet. Sous Windows, cela se règle une fois pour toutes dans le champ **Start In** du raccourci.

● Concept 4 — Deux sortes de commandes, et rien d'autre

DÉFINITION (§1.8).

Techniquement, R est un **langage d'expressions** (*expression language*) à la syntaxe très simple. Les **commandes élémentaires** consistent soit en des **expressions**, soit en des **assignments**.

	Expression	Assignment
Exemple	<code>1/x</code>	<code>y <- 1/x</code>
Est évaluée	oui	oui
Est affichée	oui , sauf si rendue explicitement invisible	non
La valeur	est perdue	est passée à une variable

C'est la distinction structurante de toute la pratique de R : **si rien ne s'affiche, c'est probablement que vous avez assigné**. Et réciproquement, un débutant qui « perd » son résultat a tapé une expression au lieu d'une assignation.

Le « **sauf si** » n'est pas anodin. « *it is evaluated, printed (**unless specifically made invisible**), and the value is lost.* » Certaines fonctions renvoient leur valeur de façon invisible — c'est pourquoi `library(stats)` n'affiche rien tout en renvoyant quelque chose.

ENRICHISSEMENT PÉDAGOGIQUE (HORS COURS) — LA VALEUR N'EST PAS TOUT À FAIT PERDUE.

La note 2 du §2.1 précise : « *Actually, it is still available as `.Last.value` **before any other statements are executed**.* » Une expression dont vous avez oublié de garder le résultat est donc récupérable, **une fois**, par `.Last.value`.

4.1 Noms : ce qui est permis, ce qui est interdit

Règle (§1.8). R est **sensible à la casse**, comme la plupart des paquets fondés sur UNIX : `A` et `a` sont **des symboles différents** et renverraient à des variables différentes.

L'ensemble des symboles utilisables dans un nom R **dépend du système d'exploitation et du pays** dans lequel R tourne (techniquement, de la **locale** en usage).

Règle (§1.8)	Détail
Caractères permis	tous les alphanumériques (dans certains pays, y compris les lettres accentuées) plus <code>.</code> et <code>_</code>
Premier caractère	doit être <code>.</code> ou une lettre
Si le nom commence par <code>.</code>	le deuxième caractère ne doit pas être un chiffre
Longueur	<i>effectively unlimited</i>
Code portable (note 1)	« <i>For portable R code (including that to be used in R packages) only A–Z, a–z, and 0–9 should be used.</i> »

COMPLÉMENT — LA FORMULATION EXACTE DE LA R LANGUAGE DEFINITION (§10.3.2).

Les identifiants ne doivent pas commencer **par un chiffre, par un tiret bas, ni par un point suivi d'un chiffre**. L'ensemble précis des caractères permis est donné par l'expression `C (isalnum(c) || c == '.' || c == '_')`. Elle ajoute deux faits utiles : les identifiants **commençant par un point ne sont pas listés par** `1s`, et `...`, `..1`, `..2`, etc. **sont spéciaux** (voir fiche 308).

Un objet peut porter un nom qui n'est pas un identifiant (*R Language Definition* §10.3.2) : on y accède alors par `get` et `assign`. Mais `get` et `assign` **ne comprennent ni l'indexation ni les fonctions de remplacement**. Les paires suivantes ne sont **pas** équivalentes :

Écriture correcte	Ce que <code>assign</code> / <code>get</code> fait à la place
<code>x\$a <- 1</code>	<code>assign("x\$a", 1)</code> crée un objet nommé <code>x\$a</code>
<code>x[[1]]</code>	<code>get("x[[1]]")</code> cherche un objet nommé <code>x[[1]]</code>
<code>names(x) <- nm</code>	<code>assign("names(x)", nm)</code> crée un objet nommé <code>names(x)</code>

4.2 Les mots réservés

FORMULE — LA LISTE COMPLÈTE (R LANGUAGE DEFINITION §10.3.3).

Les identifiants suivants ont une signification spéciale et **ne peuvent pas être utilisés comme noms d'objets** :

```
if else repeat while function for in next break
TRUE FALSE NULL Inf NaN
NA NA_integer_ NA_real_ NA_complex_ NA_character_
... ..1 ..2 etc.
```

T et **F** ne sont PAS dans cette liste — et c'est exactement le problème. Le cours y revient au §2.4 : * `TRUE` et `FALSE` sont souvent abrégés en `T` et `F`. Notez cependant que **T** et **F** sont juste des variables initialisées à `TRUE` et `FALSE` par défaut, mais ne sont pas des mots réservés et peuvent donc être écrasées par l'utilisateur. Par conséquent, vous devriez toujours utiliser `TRUE` et `FALSE`. * Voir fiche 301.

4.3 Séparer, grouper, commenter, continuer

Élément (§1.8)	Règle
Séparateur de commandes	un point-virgule <code>;</code> ou un saut de ligne
Groupement	des accolades <code>{</code> et <code>}</code> réunissent des commandes élémentaires en une expression composée
Commentaire	commence par un <code>#</code> ; <i>tout ce qui suit jusqu'à la fin de la ligne est un commentaire</i>
Commentaire — où c'est interdit (note 2)	pas à l'intérieur d'une chaîne , ni dans la liste d'arguments d'une définition de fonction
Commande incomplète	R affiche une invite différente , par défaut <code>+</code> , et continue à lire jusqu'à ce que la commande soit syntactiquement complète
Longueur d'une ligne à la console	limitée à environ 4 095 octets (pas caractères)

Le comportement des consoles au-delà de 4 095 octets n'est pas uniforme (note 3, §1.8) : certaines ne vous laisseront pas en saisir davantage ; parmi celles qui le permettent, **certaines jettent silencieusement l'excédent et d'autres l'utilisent comme début de la ligne suivante**. Trois comportements incompatibles — d'où la règle : **les commandes longues vont dans un fichier**, pas dans la console (concept 6, où la note 4 précise que les commandes d'un fichier sont *of unlimited length*).

Le piège d'espacement le plus célèbre de R (*R Language Definition* §10.3.5). Des suites de caractères d'espacement servent à **délimiter les jetons en cas d'ambiguïté** — comparez `x<-5` et `x < -5`. Le premier **assigne 5 à x**. Le second **teste si x est inférieur à -5**. Un seul espace sépare une écriture de l'autre.

La règle spéciale du `else` (*R Language Definition* §10.3.5). À l'intérieur d'une expression composée, un saut de ligne **avant `else` est ignoré** ; **au niveau le plus externe**, le saut de ligne **termine la construction `if` et un `else` qui suit provoque une erreur de syntaxe**. Ce comportement « quelque peu anormal » — le mot est du cours — existe *parce que R doit être utilisable en mode interactif* : le parseur doit décider **dès la frappe de RET** si l'expression est complète, incomplète ou invalide. Conséquence pratique : à la console, écrivez `} else {` **sur la même ligne**. Voir fiche 307.

● Concept 5 — L'aide : quatre portes, une par situation

C'est la section la plus rentable de la fiche. Le §1.3 a prévenu : « *users may need to be prepared to do a little work to find it* ». Voici l'outillage exact.

Situation	Commande	Détail du cours (§1.7)
Je connais le nom	<code>help(solve)</code> ou <code>?solve</code>	« <i>To get more information on any specific named function</i> »
Le nom contient des caractères spéciaux	<code>help("[")</code>	<i>l'argument doit être entouré de guillemets doubles ou simples, ce qui en fait une chaîne de caractères</i>
Le nom est un mot à sens syntaxique	<code>help("if")</code> , <code>help("for")</code> , <code>help("function")</code>	« <i>This is also necessary for a few words with syntactic meaning including <code>if</code> , <code>for</code> and <code>function</code> »</i>
Je cherche par sujet	<code>??solve</code> , ou <code>help.search(...)</code>	« <i>allows searching for help in various ways</i> » — <code>?help.search</code> pour les détails et d'autres exemples
Je veux voir tourner	<code>example(topic)</code>	« <i>The examples on a help topic can normally be run by</i> »
Je veux naviguer	<code>help.start()</code>	lance un navigateur web ; les pages d'aide se parcourent par hyperliens

MÉTHODE — COMMENT SE REPÉRER DANS UN DOMAINE INCONNU (§1.7).

Le lien « **Search Engine and Keywords** » de la page chargée par `help.start()` est **particulièrement utile**, car il contient une **liste de concepts de haut niveau** qui cherche parmi les fonctions disponibles. « *It can be a great way to get your bearings quickly and to understand the breadth of what R has to offer.* »

Sous UNIX, `help.start()` change le canal de l'aide : « *On UNIX, subsequent help requests are sent to the HTML-based help system* ». Après un `help.start()` , un `?solve` ne répond plus dans la console mais **dans le navigateur**. Ce n'est pas une panne.

Les guillemets (§1.7). L'une ou l'autre forme de guillemet peut être utilisée **pour échapper l'autre**, comme dans la chaîne `"It's important"` . **Notre convention est d'utiliser de préférence les guillemets doubles.**

Windows (§1.7). Les versions Windows de R ont d'autres systèmes d'aide optionnels : utilisez `?help` pour plus de détails.

Énoncé. Vous voulez comprendre la différence entre `[]` et `[[`. Vous tapez `?[]` et R proteste. Pourquoi, et comment faire ?

Étape 1 — *pourquoi ça casse.* `?[]` n'est pas une expression R valide : `[]` est un **opérateur**, pas un identifiant. Le parseur (§10.3.8 de la *R Language Definition*) le lit comme un **jeton d'indexation**, et attend un objet à indexer.

Étape 2 — *la règle du cours.* « For a feature specified by **special characters**, the argument **must be enclosed in double or single quotes**, making it a "character string" ».

Étape 3 — *l'écriture correcte.*

```
help("[")
```

Étape 4 — *la variante.* `? "["` fonctionne aussi : c'est la même chaîne, passée par l'autre forme.

Étape 5 — *généraliser.* La même règle s'applique aux mots à sens syntaxique : `help("if")`, `help("for")`, `help("function")`, `help("repeat")`. `?if` **est une erreur de syntaxe, pas une absence de documentation.**

Étape 6 — *voir tourner.* `example("[")` exécute les exemples de la page.

● Concept 6 — Sortir de la console : `source()` et `sink()`

DÉFINITION (§1.10).

Si des **commandes** sont stockées dans un **fichier externe**, disons `commands.R` dans le répertoire de travail `work`, elles peuvent être exécutées **à tout moment** dans une session R par :

```
source("commands.R")
```

Sous Windows, **Source** est aussi disponible dans le menu **File**.

DÉFINITION (§1.10).

La fonction `sink` **détourne toute la sortie ultérieure** de la console vers un fichier externe.

```
sink("record.lis") # tout ce qui s'afficherait part dans record.lis
# ... commandes ...
sink()             # retour a la console
```

La symétrie à retenir. `source()` fait **entrer** du texte dans R ; `sink()` fait **sortir** du texte de R. Ce sont les deux moitiés du même besoin : **ne pas dépendre de ce qui a été tapé à la main.**

Un `sink()` **non fermé donne l'impression que R est muet.** Toute la sortie continue de partir dans le fichier — y compris les messages qui vous diraient ce qui ne va pas. Le `sink()` **sans argument** est le remède, et il doit être écrit **au moment même** où on ouvre le premier.

CONTRÔLE — LA RAISON TECHNIQUE DE PRÉFÉRER UN FICHIER.

La note 4 du §1.10 précise que les commandes stockées dans un fichier sont « *of unlimited length* », là où une ligne de console est **limitée à ~4 095 octets** (§1.8). Le fichier n'est donc pas seulement plus propre : il lève **une limite réelle.**

● Concept 7 — Le démarrage de R : cinq étapes, dans cet ordre

C'est la partie que personne ne lit et que tout le monde finit par avoir besoin de connaître — parce qu'elle explique **pourquoi R ne se comporte pas pareil sur deux machines.**

MÉTHODE — LA SÉQUENCE COMPLÈTE (ANNEXE B.1).

Le mécanisme de démarrage est le suivant.

Ordre	Étape	Fichier cherché	Option qui la coupe
1	variables d'environnement, site	<code>R_ENVIRON</code> , sinon <code>R_HOME/etc/Renviron.site</code> (s'il existe)	<code>--no-environ</code>
1 bis	variables d'environnement, utilisateur	<code>R_ENVIRON_USER</code> , sinon <code>.Renviron</code> dans le répertoire courant puis dans le répertoire personnel , <i>dans cet ordre</i>	<code>--no-environ</code>
2	profil de site	<code>R_PROFILE</code> , sinon <code>R_HOME/etc/Rprofile.site</code>	<code>--no-site-file</code>
3	profil utilisateur	<code>R_PROFILE_USER</code> , sinon <code>.Rprofile</code> dans le répertoire courant puis dans le répertoire personnel	<code>--no-init-file</code>
4	espace de travail sauvegardé	<code>.RData</code> du répertoire courant, <i>s'il y en a un</i>	<code>--no-restore</code> , <code>-no-restore-data</code>
5	<code>.First()</code>	la fonction, si elle existe	— (dépend de 3 et 4)

Les fichiers `.Renviron` doivent contenir des lignes de la forme `name=vaLue` . Les variables que « *you might want to set* », nommément citées : `R_PAPERSIZE` (format de papier par défaut), `R_PRINTCMD` (commande d'impression par défaut), `R_LIBS` (« *specifies the list of R library trees searched for add-on packages* »).

DÉFINITION — `.FIRST()` ET `.LAST()` (ANNEXE B.1).

*Enfin, **si une fonction `.First()` existe, elle est exécutée**. Cette fonction — comme `.Last()` , **exécutée à la fin de la session R** — peut être définie dans les profils de démarrage appropriés, ou **résider dans `.RData` .***

`.First()` **peut venir de `.RData`** . C'est la conséquence la plus surprenante du tableau : l'étape 4 précède l'étape 5. Un espace de travail sauvegardé peut donc **injecter du code exécuté au démarrage**. Combiné au fait que `ls()` ne montre pas les noms commençant par un point (§1.11), cela donne un comportement **invisible et persistant**.

MÉTHODE — LE RÉFLEXE DE DIAGNOSTIC.

Devant un comportement inexplicable au démarrage, **relancez R avec `--vanilla`** : l'option combine `--no-save`, `--no-envIRON`, `--no-site-file`, `--no-init-file` et `--no-restore` (et, sous Windows, `--no-Rconsole`). Elle **désactive les cinq étapes d'un coup**. Si le problème disparaît, il vient d'un fichier de démarrage ; s'il persiste, il vient de R ou du paquet.

Voir aussi l'aide en ligne du sujet `Startup` : `help("Startup")` en donne « *a precise description* ».

Une dernière contrainte, avant tout le reste (annexe B.1). « *You need to ensure that either the environment variable `TMPDIR` is **unset** or it **points to a valid place to create temporary files and directories**.* »

● Concept 8 — Invoquer R depuis la ligne de commande

NOTATION (ANNEXE B.1).

En travaillant en ligne de commande sous UNIX ou Windows, la commande `R` peut servir à la fois à lancer le programme principal, sous la forme

```
R [options] [<infile] [>outfile]
```

*ou, via l'interface `R CMD`, comme enveloppe autour de divers outils R (par exemple pour traiter des fichiers au format de documentation R ou manipuler des paquets additionnels) **qui ne sont pas destinés à être appelés « directement »**.*

« **At the Windows command-line, `Rterm.exe` is preferred to `R`.** » C'est explicite dans le cours.

8.1 Les options — regroupées par ce qu'elles servent

Information, puis sortie immédiate

Option	Effet
<code>--help</code> , <code>-h</code>	<i>print short help message to standard output and exit successfully</i>
<code>--version</code>	information de version, puis sortie
<code>RHOME</code>	affiche le chemin du « home directory » de R. L'installation de R y met tout (exécutables, paquets, etc.), à part le script shell de façade et la page de manuel

Ce qui est lu au démarrage et écrit à la sortie — voir concept 7

Option	Effet
<code>--save</code> / <code>--no-save</code>	contrôlent la sauvegarde en fin de session. Si aucune n'est donnée en session interactive , l'utilisateur est interrogé ; en usage non interactif , l'une des deux doit être spécifiée ou impliquée par une autre option
<code>--no-envIRON</code>	ne lit aucun fichier utilisateur de variables d'environnement
<code>--no-site-file</code>	ne lit pas le profil de site
<code>--no-init-file</code>	ne lit pas le profil utilisateur
<code>--restore</code> / <code>--no-restore</code> / <code>--no-restore-data</code>	restauration de <code>.RData</code> . Le défaut est de restaurer . <code>--no-restore</code> implique toutes les options <code>--no-restore-*</code> spécifiques
<code>--no-restore-history</code>	l'historique (<code>.Rhistory</code> , ou <code>R_HISTFILE</code>). Le défaut est de restaurer
<code>--no-Rconsole</code>	(Windows) empêche le chargement du fichier <code>Rconsole</code>
<code>--vanilla</code>	combine <code>--no-save</code> , <code>--no-envIRON</code> , <code>--no-site-file</code> , <code>--no-init-file</code> , <code>--no-restore</code> (+ <code>--no-Rconsole</code> sous Windows)

D'où vient le code

Option	Effet
<code>-f fichier</code> , <code>--file=fichier</code>	(pas <code>Rgui.exe</code>) prend l'entrée dans le fichier ; <code>-</code> signifie l'entrée standard. Implique <code>--no-save</code> sauf si <code>--save</code> a été posé. Sur système Unix, <i>éviter les métacaractères du shell dans le nom (les espaces sont permis)</i>
<code>-e expression</code>	(pas <code>Rgui.exe</code>) utilise l'expression comme ligne d'entrée. Plusieurs <code>-e</code> possibles, mais pas avec <code>-f</code> / <code>--file</code> . Implique <code>--no-save</code> sauf <code>--save</code> . Limite de 10 000 octets sur la longueur totale des expressions
<code>--encoding=enc</code>	encodage supposé pour l'entrée depuis la console ou <code>stdin</code> ; doit être un encodage connu de <code>iconv</code>

Verbo­si­té

Option	Effet
<code>--quiet</code> , <code>--silent</code> , <code>-q</code>	pas de message initial de copyright et de bienvenue ; en outre pose <code>options("quiet")</code> à <code>TRUE</code> — que d'autres fonctions R peuvent consulter
<code>--no-echo</code> , <code>-s</code>	« <i>make R run as quietly as possible</i> ». Implique <code>--quiet</code> et <code>--no-save</code> . Destinée aux programmes qui utilisent R pour calculer des résultats pour eux
<code>--verbose</code>	plus d'informations sur la progression ; pose l'option <code>verbose</code> de R à <code>TRUE</code> — le code R utilise cette option pour contrôler l'affichage des messages de diagnostic

Cas particuliers

Option	Effet
<code>--interactive</code>	(UNIX) affirmer que R tourne bien de façon interactive même si l'entrée a été redirigée . Le défaut est de déduire l'interactivité si et seulement si <code>stdin</code> est connecté à un terminal ou à un pty . <code>-e</code> , <code>-f</code> ou <code>--file</code> affirment un usage non interactif même si <code>--interactive</code> est donné — et cela n'active pas l'édition de ligne de commande
<code>--no-readline</code>	(UNIX) coupe l'édition de ligne via readline . Utile depuis Emacs avec ESS . Affecte aussi l'expansion du tilde : voir l'aide de <code>path.expand</code>
<code>--ess</code>	(Windows) prépare <code>Rterm</code> pour <code>R-inferior-mode</code> d'ESS : affirme l'interactivité (sans l'éditeur de ligne) et pas de tampon sur <code>stdout</code>
<code>--gui=type</code> , <code>-g type</code>	(UNIX) interface graphique : <code>x11</code> (défaut) et <code>Tk</code> si le support Tcl/Tk est disponible. Par rétrocompatibilité, <code>x11</code> et <code>tk</code> sont acceptés
<code>--arch=nom</code>	(UNIX) lance la sous-architecture indiquée
<code>--debugger=nom</code> , <code>-d nom</code>	(UNIX) lance R sous le débogueur. Pour la plupart des débogueurs — les exceptions sont <code>valgrind</code> et les versions récentes de <code>gdb</code> — les options de ligne de commande suivantes sont ignorées et doivent être données au lancement de l'exécutable depuis l'intérieur du débogueur
<code>--args</code>	« <i>does nothing except cause the rest of the command line to be skipped</i> » — pour récupérer les valeurs avec <code>commandArgs(TRUE)</code>

Mémoire — « for expert use only »

Option	Défaut	Variable d'environnement
<code>--min-vsize=N</code> (tas de vecteurs, en octets)	6 Mo	<code>R_VSIZE</code>
<code>--min-nspace=N</code> (cellules cons, en nombre)	350 k	<code>R_NSIZ</code>
<code>--max-ppsize=N</code> (pile de protection des pointeurs)	10 000 , maximum accepté 100 000	—

Le suffixe `M` spécifie des mégaoctets ou des millions de cellules respectivement. Voir l'aide du sujet `Memory`.
« Users will not normally need to use these unless they are trying to limit the amount of memory used by R. »

La redirection ne lève pas la limite de ligne. « *Note that input and output can be redirected in the usual way (using `<` and `>`), but **the line length limit of 4 095 bytes still applies.*** » Et : « *Warning and error messages are sent to the **error channel** (`stderr`)* » — ce qui explique qu'un `> sortie.txt` ne capture **pas** les erreurs.

● Concept 9 — `R CMD` : la boîte à outils hors session

NOTATION (ANNEXE B.1).

*La commande `R CMD` permet d'invoquer divers outils utiles avec R, **mais non destinés à être appelés « directement »**. La forme générale est :*

`R CMD` commande args

Outil	Ce qu'il fait
BATCH	<i>run R in batch mode.</i> Lance <code>R --restore --save</code> avec éventuellement d'autres options (voir <code>? BATCH</code>)
COMPILE	(UNIX) compile des fichiers C, C++, Fortran... pour usage avec R
SHLIB	construit une bibliothèque partagée pour le chargement dynamique
INSTALL / REMOVE	installe / supprime des paquets additionnels
build	empaquette un paquet additionnel
check	vérifie un paquet additionnel
LINK	(UNIX) façade pour créer des programmes exécutables
Rprof	post-traite les fichiers de profilage R (voir fiche 316)
Rdconv , Rd2txt	convertit le format Rd vers HTML, LaTeX, texte brut, et en extrait les exemples . <code>Rd2txt</code> est un raccourci pour <code>Rd2conv -t txt</code>
Rd2pdf	convertit du Rd en PDF
Stangle / Sweave	extrait le code S/R d'une vignette / traite une vignette
Rdiff	<i>diff</i> de sortie R en ignorant les en-têtes
config	obtient l'information de configuration
javareconf	(UNIX) met à jour les variables de configuration Java
rtags	(UNIX) crée des fichiers de tags style Emacs à partir de fichiers C, R et Rd
open	(Windows) ouvre un fichier via les associations Windows
texify	(Windows) traite des fichiers (La)TeX avec les styles de R

`R CMD commande --help` donne les informations d'usage de chaque outil.

Deux règles d'or de `R CMD`, faciles à manquer.

1. Vous pouvez placer `--arch=`, `--no-envIRON`, `--no-init-file`, `--no-site-file` et `--vanilla` **entre** `R` **et** `CMD` : elles affectent tous les processus `R` lancés par les outils. Ici `--vanilla` **équivalent à** `--no-envIRON --no-site-file --no-init-file` (les deux autres composantes n'ont pas de sens hors session).
2. `R CMD` **n'utilise de lui-même aucun fichier de démarrage de `R`** (en particulier ni les `Renviron` utilisateur ni ceux de site), *et **tous les processus `R` lancés par ces outils — sauf `BATCH` — utilisent `--no-restore`**.*

Autrement dit : **ce qui marche dans votre session ne marche pas forcément sous `R CMD check`**, précisément parce que votre `.Rprofile` n'y est pas lu. C'est la cause n° 1 des « ça marche chez moi » en développement de paquets.

● Concept 10 — Scripter : `BATCH`, `Rscript`, et la question de `stdin`

MÉTHODE — LA RECOMMANDATION DU COURS (ANNEXE B.4).

*Si vous voulez simplement exécuter un fichier `foo.R` de commandes `R`, **la façon recommandée est `R CMD BATCH foo.R`**. Pour le lancer en arrière-plan ou comme travail par lot, utilisez les facilités du système : par exemple, dans la plupart des shells Unix, `R CMD BATCH foo.R &` lance un travail d'arrière-plan.*

10.1 Passer des arguments — les deux voies

Voie 1 — `R CMD BATCH`. On peut passer des paramètres via des arguments supplémentaires sur la ligne de commande (le quotage exact dépend du shell utilisé) :

```
R CMD BATCH "--args arg1 arg2" foo.R &
```

ce qui passe des arguments à un script, récupérables comme **vecteur de caractères** par :

```
args <- commandArgs(TRUE)
```


Voie 2 — `Rscript` , « **this is made simpler by** ». La façade alternative `Rscript` s'invoque par :

```
Rscript foo.R arg1 arg2
```

10.2 Écrire un script exécutable

Cela peut aussi servir à écrire des fichiers de script exécutables comme celui-ci (au moins sur systèmes Unix, et dans certains shells Windows) :

```
#!/path/to/Rscript
args <- commandArgs(TRUE)
...
q(status = <exit status code>)
```

Si ceci est saisi dans un fichier texte `runfoo` **rendu exécutable** (par `chmod 755 runfoo`), il peut être invoqué avec différents arguments par `runfoo arg1 arg2` .

Si vous ne voulez pas coder en dur le chemin de `Rscript` mais qu'il est dans votre `PATH` — **ce qui est normalement le cas pour une installation R, sauf sous Windows**, mais les utilisateurs macOS peuvent avoir à ajouter `/usr/local/bin` — utilisez :

```
#!/usr/bin/env Rscript
...
```

« **At least in Bourne and bash shells, the `#!` mechanism does not allow extra arguments like `#!/usr/bin/env Rscript --vanilla` .** » Le shebang n'accepte **pas** d'argument supplémentaire après `env Rscript` . C'est une limitation du shell, pas de R.

« This writes R output to `stdout` and `stderr` , and this can be redirected in the usual way for the shell running the command. »

10.3 Le piège de `stdin()` — la subtilité qui fait échouer les scripts

PROBLÈME.

« One thing to consider is what `stdin()` refers to. »

Il est courant d'écrire des scripts R avec des segments comme :

```
chem <- scan(n = 24)
2.90 3.10 3.40 3.40 3.70 3.70 2.80 2.50 2.40 2.40 2.70 2.20
5.28 3.37 3.03 3.03 28.95 3.77 3.40 2.20 3.50 3.60 3.70 3.70
```

et `stdin()` renvoie au **fichier de script**, pour permettre cet usage traditionnel. Si vous voulez désigner l'**entrée standard du processus**, utilisez `"stdin"` comme **connexion de fichier**, par exemple `scan("stdin", ...)`.

Écriture	Ce qu'elle lit
<code>scan(n = 24) / stdin()</code>	les lignes suivantes du fichier de script lui-même
<code>scan("stdin", ...)</code>	l'entrée standard du processus (le tube, le clavier)

Une autre façon d'écrire des scripts exécutables (suggérée par François Pinard) est d'utiliser un **document en ligne** (here document) :

```
#!/bin/sh
# des variables d'environnement peuvent etre posees ici
R --no-echo [autres options] <<EOF

le programme R vient ici...

EOF
```

mais `ici stdin()` renvoie à la source du programme et `"stdin"` ne sera pas utilisable. Les deux techniques ont donc des sémantiques d'entrée **opposées** : c'est le point à vérifier avant de porter un script d'une forme à l'autre.

Deux dernières notes (annexe B.4). Les scripts courts peuvent être passés à `Rscript` en ligne de commande via le drapeau `-e`. **Les scripts vides ne sont pas acceptés.** Et : sur un système Unix, le nom du fichier d'entrée (tel que `foo.R`) **ne devrait contenir ni espaces ni métacaractères du shell.**

● Concept 11 — L'éditeur de ligne de commande

DÉFINITION (ANNEXE C.1).

Quand la bibliothèque **GNU readline** est disponible au moment où R est configuré pour la compilation sous UNIX, **un éditeur de ligne de commande intégré** permettant le rappel, l'édition et la resoumission des commandes antérieures est utilisé.

- D'autres versions de readline existent et peuvent être utilisées — « *this is most common on macOS* ».
- **Savoir laquelle vous avez** : `extSoftVersion()` dans une session R.
- **Le désactiver** (utile avec ESS) : option de démarrage `--no-readline`.
- **Windows** : édition « *somewhat simpler* » — voir `Console` dans le menu `Help` de l'interface, et le fichier `README.Rterm` pour `Rterm.exe`.

Notation (annexe C.1). *Les caractères **Control**, comme Control-m, s'obtiennent en maintenant CTRL pendant qu'on presse `m` ; on les écrit `C-m`. Les caractères **Meta**, comme Meta-b, se tapent en maintenant META et en pressant `b` ; on les écrit `M-b`. Si votre terminal n'a pas de touche META activée, vous pouvez taper les caractères Meta par une **séquence de deux caractères commençant par ESC** — ainsi `M-b` peut se taper `ESC b`.*

« **Note that case is significant for Meta characters.** » `M-l` et `M-L` ne sont pas la même chose.

« *Some but not all versions of readline will recognize **resizing of the terminal window** so this is best avoided.* »

Le fonctionnement (annexe C.2). *R* garde un historique des lignes de commande que vous tapez, **y compris les lignes erronées**. En édition à la Emacs, **toute frappe ordinaire insère les caractères** dans la commande en cours, en décalant vers la droite. En mode `vi`, le mode insertion se démarre par `M-i` ou `M-a` et se termine par un nouveau `ESC`. **Le défaut est le style Emacs**, et c'est le seul décrit. **Presser RET à tout moment resoumet la commande.**

11.1 Le tableau complet des raccourcis (annexe C.3)

Rappel de commandes et déplacement vertical

Raccourci	Action
<code>C-p</code>	commande précédente (en arrière dans l'historique)
<code>C-n</code>	commande suivante
<code>C-r</code> texte	trouve la dernière commande contenant la chaîne. Annulable par <code>C-g</code> , et sur certaines versions de R par <code>C-c</code>

Sur la plupart des terminaux, les **flèches haut et bas** remplacent `C-p` et `C-n`.

Déplacement horizontal du curseur

Raccourci	Action
<code>C-a</code>	aller au début de la commande
<code>C-e</code>	aller à la fin de la ligne
<code>M-b</code> / <code>M-f</code>	reculer / avancer d'un mot
<code>C-b</code> / <code>C-f</code>	reculer / avancer d'un caractère

Sur la plupart des terminaux, les **flèches gauche et droite** remplacent `C-b` et `C-f`.

Édition et resoumission

Raccourci	Action
<code>texte</code>	insérer le texte au curseur
<code>C-f texte</code>	ajouter le texte après le curseur
<code>DEL</code>	effacer le caractère précédent (à gauche du curseur)
<code>C-d</code>	effacer le caractère sous le curseur
<code>M-d</code>	effacer le reste du mot sous le curseur, et le « sauver »
<code>C-k</code>	effacer du curseur à la fin de la commande, et le « sauver »
<code>C-y</code>	insérer (<i>yank</i>) le dernier texte « sauvé » ici
<code>C-t</code>	transposer le caractère sous le curseur avec le suivant
<code>M-l</code>	passer le reste du mot en minuscules
<code>M-c</code>	passer le reste du mot en majuscules
<code>RET</code>	resoumettre la commande à R

« *The final RET terminates the command line editing sequence.* »

Personnalisation (annexe C.3). *Les associations de touches de readline peuvent être personnalisées de la façon habituelle via un fichier `~/.inputrc`. Ces personnalisations peuvent être **conditionnées à l'application R**, en incluant une section comme :*

```
$if R
    "\C-xd": "q('no')\n"

$endif
```

CE QUE FAIT CET EXEMPLE.

Il associe `C-x d` à la frappe littérale de `q('no')` suivie d'un retour chariot — **quitter sans sauvegarder, en deux touches**. Et le bloc `$if R ... $endif` garantit que cette association **n'existe que dans R**, pas dans les autres programmes qui utilisent readline.

Le trio à mémoriser en premier, parce qu'il représente 90 % de l'usage réel : **C-r** (retrouver une commande par son contenu), **C-a** (revenir au début pour corriger un nom de fonction), **C-k** (couper la fin d'une ligne à refaire).

● La session de l'annexe A, expliquée pas à pas

L'annexe A est la seule partie du cours qui soit un **script à taper**. « *Many features of the system will be unfamiliar and puzzling at first, but this puzzlement will soon disappear.* » Elle est reproduite ici découpée en actes, avec **ce que chaque ligne enseigne**. Les fiches 301 à 313 y reviendront ; l'objectif ici est de **reconnaître la forme d'une session R**.

```
help.start()      # interface HTML de l'aide, dans un navigateur
x <- rnorm(50)
y <- rnorm(x)      # deux vecteurs pseudo-aleatoires normaux
plot(x, y)        # nuage de points ; une fenetre graphique apparait
ls()              # quels objets sont dans l'espace de travail ?
rm(x, y)          # nettoyage
```

rnorm(x) n'est pas une faute de frappe. **rnorm(50)** tire 50 valeurs ; **rnorm(x)** en tire autant que **x** a d'éléments. C'est un idiome R : beaucoup de fonctions acceptent un vecteur là où on attend une longueur. Le cours ne le commente pas — il le fait.

```
x <- 1:20          # x = (1, 2, ..., 20)
w <- 1 + sqrt(x)/2  # un vecteur de POIDS : des ecarts-types
dummy <- data.frame(x = x, y = x + rnorm(x)*w)
dummy              # un data frame a deux colonnes ; le regarder
```

Étape — lire le modèle simulé. La colonne **y** vaut **x** plus un bruit normal d'écart-type **w**, lui-même croissant en $\sqrt{x}$. Les données sont donc **hétéroscédastiques par construction** — ce qui va être le sujet de tout l'acte.

```
fm <- lm(y ~ x, data = dummy)
summary(fm)
```

*« Fit a simple linear regression and look at the analysis. **With y to the left of the tilde, we are modelling y dependent on x.** »* Voir fiche 312 pour la syntaxe des formules.

```
fm1 <- lm(y ~ x, data = dummy, weight = 1/w^2)
summary(fm1)
```

« *Since we know the standard deviations, we can do a weighted regression.* » Le poids est **l'inverse de la variance** — c'est la définition des moindres carrés pondérés.

```
attach(dummy)          # rend les colonnes du data frame visibles comme variables
lrf <- lowess(x, y)      # une regression locale non parametrique
plot(x, y)              # nuage standard
lines(x, lrf$y)         # y ajouter la regression locale
abline(0, 1, lty = 3)   # LA VRAIE droite : ordonnee 0, pente 1
abline(coef(fm))        # droite non ponderee
abline(coef(fm1), col = "red") # droite ponderee
detach()                # retirer le data frame du chemin de recherche
```

Ce que cet acte démontre. Trois estimations de la même relation sont superposées **au vrai modèle** (`abline(0, 1)`), qu'on ne connaît que parce que les données sont simulées. C'est la manière la plus économique de voir **ce que la pondération apporte**.

```
plot(fitted(fm), resid(fm),
     xlab = "Fitted values",
     ylab = "Residuals",
     main = "Residuals vs Fitted")
```

« *A standard regression diagnostic plot **to check for heteroscedasticity. Can you see it?*** » La question du cours est rhétorique : **oui**, puisque `w` croît avec `x`.

```
qqnorm(resid(fm), main = "Residuals Rankit Plot")
```

*« A normal scores plot to check for **skewness, kurtosis and outliers. (Not very useful here.)** »*

```
rm(fm, fm1, lrf, x, dummy) # nettoyage
```

« The next section will look at data from the classical experiment of **Michelson to measure the speed of light**. This dataset is available in the `morley` object, **but we will read it to illustrate the `read.table` function**. »

```
filepath <- system.file("data", "morley.tab", package = "datasets")
filepath          # le chemin du fichier de donnees
file.show(filepath) # optionnel : regarder le fichier
mm <- read.table(filepath)
mm
```

« There are **five experiments** (column `Expt`) and each has **20 runs** (column `Run`) and `sl` is the recorded speed of light, suitably coded. »

```
mm$Expt <- factor(mm$Expt)
mm$Run  <- factor(mm$Run)    # transformer Expt et Run en FACTEURS
```

C'est l'étape décisive. `Expt` contient les nombres 1 à 5, mais ce sont des **étiquettes**, pas des quantités. Sans `factor()`, R traiterai « expérience 4 » comme « deux fois expérience 2 ». Voir fiche 303.

```
attach(mm)          # rend le data frame visible en position 2 (le default)
plot(Expt, Speed, main = "Speed of Light Data", xlab = "Experiment No.")
```

« Compare the five experiments with **simple boxplots**. » Remarquez qu'on a appelé `plot`, pas `boxplot` : parce que `Expt` **est un facteur**, `plot` produit des boîtes à moustaches. C'est le **dispatch S3** en action (fiche 310).

```
fm <- aov(Speed ~ Run + Expt, data = mm)
summary(fm)          # analyse en BLOCS ALEATOIRES
fm0 <- update(fm, . ~ . - Run) # le sous-modele sans « runs »
anova(fm0, fm)        # les comparer par une analyse de variance formelle
detach()
rm(fm, fm0)
```


Ce que cet acte démontre. `update()` construit un modèle à **partir d'un autre**, par différence : `. ~ . - Run` signifie « même réponse, mêmes prédicteurs, **moins** `Run` ». C'est la traduction directe de la philosophie du §1.3 : le résultat est un **objet**, donc il est **modifiable**.

```
x <- seq(-pi, pi, len = 50) # 50 valeurs equidistantes sur [-pi, pi]
y <- x                      # y est le meme
f <- outer(x, y, function(x, y) cos(y)/(1 + x^2))
```

« `f` is a **square matrix**, with rows and columns indexed by `x` and `y` respectively, of values of the function $\cos(y)/(1 + x^2)$. » `outer` évalue la fonction sur **toutes les paires** — voir fiche 304.

```
oldpar <- par(no.readonly = TRUE) # SAUVER les parametres graphiques
par(pty = "s")                  # region de trace « carree »
contour(x, y, f)
contour(x, y, f, nlevels = 15, add = TRUE) # plus de lignes pour plus de detail
fa <- (f - t(f))/2               # la partie « antisymetrique » de f ; t() = transposee
contour(x, y, fa, nlevels = 15)
par(oldpar)                     # RESTAURER les anciens parametres
image(x, y, f)
image(x, y, fa)
objects(); rm(x, y, f, fa)
```

MÉTHODE — L'IDIOME OLDPAR.

`par(no.readonly = TRUE)` renvoie **les paramètres modifiables**, qu'on range ; `par(oldpar)` les remet. **Modifier** `par()` **sans le sauver contamine toutes les figures suivantes de la session**. Voir fiche 313.

« *R can do complex arithmetic, also.* »

```
th <- seq(-pi, pi, len = 100)
z <- exp(1i*th)          # 1i est le nombre complexe i
```

```
par(pty = "s")
plot(z, type = "l")      # tracer des arguments complexes = imaginaire contre reel
```

« *This should be **a circle**.* »

```
w <- rnorm(100) + rnorm(100)*1i
```

« *Suppose we want to **sample points within the unit circle**. One method would be to take complex numbers with **standard normal real and imaginary parts**...* »

```
w <- ifelse(Mod(w) > 1, 1/w, w)
```

« *...and to **map any outside the circle onto their reciprocal**.* »

```
plot(w, xlim = c(-1,1), ylim = c(-1,1), pch = "+", xlab = "x", ylab = "y")
lines(z)
```

« ***All points are inside the unit circle, but the distribution is not uniform.*** »

```
w <- sqrt(runif(100))*exp(2*pi*runif(100)*1i)
plot(w, xlim = c(-1,1), ylim = c(-1,1), pch = "+", xlab = "x", ylab = "y")
lines(z)
```

« *The second method uses the uniform distribution. **The points should now look more evenly spaced over the disc.*** »

ENRICHISSEMENT PÉDAGOGIQUE (HORS COURS) — POURQUOI LA RACINE CARRÉE.

Le cours donne les deux méthodes sans les démontrer. La raison est géométrique : l'aire d'un disque de rayon r croît comme r^2 , donc pour une densité **uniforme en aire**, le rayon doit suivre la loi de fonction de répartition $F(r) = r^2$ sur $[0, 1]$. Par inversion, $r = \sqrt{U}$ avec U uniforme — d'où le `sqrt(runif(100))`. **Sans la racine, les points s'accumuleraient au centre.** Ce raisonnement n'est pas dans *An Introduction to R* ; il explique le résultat que le cours se contente de constater.

```
rm(th, w, z)
q()
```

« *Quit the R program. You will be asked if you want to save the R workspace, and **for an exploratory session like this, you probably do not want to save it.*** »

La leçon finale de l'annexe A : la session se termine par la **réponse** `no` . La sauvegarde de l'espace de travail n'est **pas** le comportement par défaut souhaitable — elle sert à interrompre un travail long, pas à archiver une exploration.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« pourquoi rien ne s'affiche ? »	assignation contre expression (§1.8) — une assignation n'imprime pas
« pourquoi le <code>+</code> au lieu du <code>></code> ? »	commande incomplète : parenthèse, guillemet ou accolade non fermée
« mes objets ont disparu »	répertoire de lancement et <code>.RData</code> (§1.11), ou <code>--no-restore</code> / <code>--vanilla</code>
« ça marche chez moi, pas chez lui »	séquence de démarrage (annexe B.1) — tester avec <code>--vanilla</code>
« ça marche en session, pas sous <code>R CMD check</code> »	<code>R CMD</code> n'utilise aucun fichier de démarrage et impose <code>--no-restore</code>
« <code>?</code> renvoie une erreur de syntaxe »	nom à caractères spéciaux ou mot à sens syntaxique → <code>help("...")</code>
« comment récupérer les arguments du script ? »	<code>commandArgs(TRUE)</code> , précédé de <code>--args</code> en <code>BATCH</code> , direct avec <code>Rscript</code>
« le script lit les mauvaises données »	<code>stdin()</code> = le fichier de script ; l'entrée du processus est <code>"stdin"</code>
« le fichier de sortie ne contient pas l'erreur »	<i>warning and error messages are sent to</i> <code>stderr</code>
« <code>x<-5</code> ou <code>x < -5</code> ? »	espacement (<i>R Language Definition</i> §10.3.5)
« <code>T</code> a été redéfini »	<code>T</code> / <code>F</code> sont des variables , pas des mots réservés → écrire <code>TRUE</code> / <code>FALSE</code>

Comment résoudre ce type d'exercice

Protocole « démarrer un nouveau projet » — 5 étapes.

1. **Créer un répertoire dédié** et s'y placer (`mkdir work` ; `cd work`). Sous Windows : champ **Start In** du raccourci.
2. Lancer R **depuis ce répertoire** — c'est ce qui lie le projet à son `.RData`.
3. Travailler ; **vérifier périodiquement** `ls()` pour savoir ce que contient l'espace de travail.

4. **Nettoyer avec** `rm()` ce qui est intermédiaire, *avant* de quitter — ce qu'on ne nettoie pas sera rechargé demain.
5. `q()`, puis décider **consciemment** : `yes` si le travail est à reprendre, `no` si c'était exploratoire.

Protocole « R se comporte anormalement » — 4 étapes.

1. Relancer avec `--vanilla`. Le problème disparaît → il vient d'un fichier de démarrage ; il persiste → il vient de R ou d'un paquet.
2. Si c'est un fichier de démarrage, remonter **la séquence dans l'ordre** : `.Renviron` (site puis utilisateur), `Rprofile.site`, `.Rprofile`, `.RData`, `.First()`.
3. Ne pas oublier que `.RData` **peut contenir** `.First()` et que `ls()` **ne montre pas** les noms commençant par un point → `ls(all.names = TRUE)`.
4. Consulter `help("Startup")` pour la description précise.

Protocole « transformer une session en script » — 5 étapes.

1. Sortir les commandes de la console vers un fichier `.R` — la limite de 4 095 octets par ligne **disparaît**.
2. Le tester dans une session propre par `source("script.R")`.
3. Le passer hors session : `R CMD BATCH script.R` (recommandé par le cours) ou `Rscript script.R`.
4. Paramétrer : `commandArgs(TRUE)`, avec `"--args a b"` en `BATCH`, ou directement `Rscript script.R a b`.
5. Rendre exécutable si besoin : shebang `#!/usr/bin/env Rscript`, `chmod 755`, sortie par `q(status = ...)` pour que le shell connaisse le code de retour.

● Common mistakes

Erreur	Correction
Croire que <code>cancel</code> annule la sauvegarde	<code>cancel</code> revient dans la session — on ne quitte pas
Lancer R depuis n'importe où et s'étonner de ne pas retrouver ses objets	<code>.RData</code> est relu du répertoire courant ; un projet = un dossier
Chercher <code>.RData</code> dans l'explorateur	le point initial le rend invisible sur UNIX, macOS et Windows
Attendre un affichage après une assignation	une assignation n'imprime rien ; c'est le comportement correct
Croire la valeur d'une expression définitivement perdue	<code>.Last.value</code> la garde — avant toute autre instruction
Écrire <code>x<-5</code> en pensant comparer	c'est une assignation ; la comparaison est <code>x < -5</code>
Utiliser <code>T</code> et <code>F</code>	ce sont des variables réassignables : écrire <code>TRUE</code> et <code>FALSE</code>
Taper <code>?[</code> ou <code>?if</code>	caractères spéciaux et mots syntaxiques → <code>help("[")</code> , <code>help("if")</code>
Croire que <code>?</code> est cassé après <code>help.start()</code>	sous UNIX, l'aide part désormais dans le navigateur
Coller une commande de plus de 4 095 octets dans la console	selon la console : refus, troncature silencieuse , ou report sur la ligne suivante → passer par un fichier
Oublier de refermer <code>sink()</code>	R paraît muet ; tout part dans le fichier, erreurs comprises
Attendre les erreurs dans <code>> sortie.txt</code>	avertissements et erreurs vont sur <code>stderr</code>
Croire que <code>R CMD check</code> lit votre <code>.Rprofile</code>	« <i>R CMD does not of itself use any R startup files</i> » — et impose <code>-no-restore</code>
Écrire <code>#!/usr/bin/env Rscript --vanilla</code>	le mécanisme <code>#!</code> n'accepte pas cet argument supplémentaire
Utiliser <code>scan(n = 24)</code> en attendant l'entrée du tube	<code>stdin()</code> désigne le fichier de script ; pour le processus, <code>scan("stdin", ...)</code>
Utiliser <code>"stdin"</code> dans la forme <i>here document</i>	il n'y est pas utilisable ; <code>stdin()</code> y désigne la source du programme
Modifier <code>par()</code> sans sauvegarder	contamine toute la session → <code>oldpar <- par(no.readonly = TRUE)</code>

Erreur	Correction
Traiter une colonne d'étiquettes numériques comme une quantité	<code>factor()</code> — sinon « expérience 4 » vaut deux fois « expérience 2 »
Passer <code>assign("x\$a", 1)</code> pour <code>x\$a <- 1</code>	<code>assign</code> / <code>get</code> ne reconnaissent ni l'indexation ni les fonctions de remplacement
Redimensionner la fenêtre du terminal en cours d'édition	<i>some but not all versions of readline</i> le gèrent — à éviter

Ultimate Review

La phrase qui résume la philosophie. « *R will give **minimal output** and **store the results in a fit object** for subsequent interrogation by further R functions.* » Peu de texte, beaucoup d'objet.

Deux sortes de commandes. **Expression** → évaluée, **affichée** (sauf invisible), valeur **perdue** (récupérable une fois par `.Last.value`). **Assignment** → évaluée, **rangée**, **rien d'affiché**.

Le cycle de l'espace de travail. `ls()` / `objects()` listent (**pas** les noms commençant par un point) · `rm()` efface · à la sortie `yes` écrit `.RData` (objets) et `.Rhistory` (commandes) · au démarrage suivant **dans le même répertoire**, les deux sont relus. **Un dossier par analyse.**

Noms. Sensibles à la casse · alphanumériques + `.` + `_` · commencent par une lettre ou `.` · si `.`, le 2^e caractère **n'est pas un chiffre** · code portable : **A–Z a–z 0–9 seulement**. **Mots réservés**, en trois groupes : `if else repeat while function for in next break` · `TRUE FALSE NULL Inf NaN` · `NA NA_integer_ NA_real_ NA_complex_ NA_character_` · plus `...` `..1` `..2` · **T et F n'en sont pas.**

Syntaxe. Séparateurs `;` ou saut de ligne · groupement `{ }` · commentaire `#` (jamais dans une chaîne ni dans la liste d'arguments d'une définition de fonction) · continuation `+` · **4 095 octets** par ligne de console · `x<-5` ≠ `x < -5` · `} else {` sur la même ligne au niveau externe.

Les quatre portes de l'aide. `?nom` / `help(nom)` · `help("[[")` pour les caractères spéciaux et les mots syntaxiques · `??` / `help.search()` pour chercher · `example(topic)` pour faire tourner · `help.start()` pour naviguer, avec le lien **Search Engine and Keywords**.

Entrées-sorties hors console. `source("f.R")` fait entrer (**longueur illimitée**) · `sink("f")` / `sink()` fait sortir.

La séquence de démarrage. `Renviron` (site : `R_ENVIRON` sinon `R_HOME/etc/Renviron.site` ; utilisateur : `R_ENVIRON_USER` sinon `.Renviron` **courant puis personnel**) → `Rprofile.site` (`R_PROFILE`) → `.Rprofile` (`R_PROFILE_USER`) → `.RData` → `.First()` · Fin de session : `.Last()` · Variables utiles : `R_PAPERSIZE` , `R_PRINTCMD` , `R_LIBS` · `TMPDIR` **doit être non défini ou valide.**

L'option de diagnostic. `--vanilla` = `--no-save` + `--no-enviro` + `--no-site-file` + `--no-init-file` + `--no-restore` (+ `--no-Rconsole` sous Windows). Entre `R` et `CMD`, elle vaut `--no-enviro --no-site-file --no-init-file`.

Options à connaître. `-f` / `--file` et `-e` impliquent `--no-save` (`-e` : 10 000 octets max, incompatible avec `-f`) · `--no-echo` / `-s` implique `--quiet` et `--no-save` · défauts mémoire **6 Mo / 350 k** / `--max-ppsize` **10 000** (max 100 000) · `--args` fait **ignorer le reste de la ligne**.

`R CMD` . `BATCH` (= `R --restore --save`), `INSTALL`, `REMOVE`, `build`, `check`, `SHLIB`, `COMPILE`, `LINK`, `Rprof`, `Rdconv` / `Rd2txt` / `Rd2pdf`, `Sweave` / `Stangle`, `Rdiff`, `config` . **Aucun fichier de démarrage n'est lu** ; `--no-restore` **partout sauf** `BATCH` .

Scripter. `R CMD BATCH foo.R` (recommandé) · `R CMD BATCH "--args a b" foo.R &` · `Rscript foo.R a b` · `commandArgs(TRUE)` → vecteur de caractères · shebang `#!/usr/bin/env Rscript` (**pas d'argument supplémentaire**) · `chmod 755` · `q(status = ...)` · `stdin()` = **le fichier de script**, `"stdin"` = l'entrée du processus, **sauf en here document** où c'est l'inverse · pas de script vide, pas d'espace ni de métacaractère dans le nom de fichier.

Éditeur de ligne. `extSoftVersion()` pour la version · `--no-readline` pour couper · `C-p` / `C-n` historique, `C-r` **texte** recherche (annulée par `C-g`), `C-a` / `C-e` début/fin, `M-b` / `M-f` par mot, `C-k` couper la fin, `C-y` coller, `M-d` couper le mot, `C-t` transposer, `M-l` / `M-c` casse, RET resoumettre. Personnalisation : `~/.inputrc`, section `$if R ... $endif` . **La casse compte pour les Meta.**

Les chiffres du chapitre. ~25 paquets standard et recommandés livrés avec `R` · **4 095** octets par ligne de console · **10 000** octets pour les `-e` cumulés · défauts de GC **6 Mo** et **350 k** · `--max-ppsize` **10 000**, maximum **100 000** · annexe A : **50** points simulés, **5** expériences × **20** essais de Michelson, grilles de **50** et **100** points, `nlevels = 15` .

Active Recall

« *In S a statistical analysis is normally done as **a series of steps, with intermediate results being stored in objects**. Thus whereas **SAS and SPSS will give copious output** from a regression or discriminant analysis, **R will give minimal output and store the results in a fit object** for subsequent interrogation by further R functions.* »

La conséquence pratique : ce que `R` vous montre n'est pas ce qu'il a calculé. Le résultat d'un `lm()` est un **objet** qu'on interroge ensuite par `summary()`, `coef()`, `resid()`,

`fitted()` , `anova()` , `update()` . Un débutant qui cherche « le listing » cherche la mauvaise chose.

« The term "environment" is intended to characterize it as **a fully planned and coherent system**, rather than **an incremental accretion of very specific and inflexible tools**, as is frequently the case with other data analysis software. »

La conséquence est la **transférabilité** : `length()` , `names()` , `[` , `summary()` ne sont pas des commandes attachées à un type de données, ce sont des mécanismes généraux. Ce qu'on apprend sur un objet vaut pour les autres.

« If an expression is given as a command, it is **evaluated, printed (unless specifically made invisible), and the value is lost**. An assignment **also evaluates an expression and passes the value to a variable but the result is not automatically printed**. »

Donc : `1/x` **affiche** les inverses et **ne les garde pas** ; `y <- 1/x` **ne montre rien et les garde**. La valeur d'une expression reste accessible **une fois** via `.Last.value` , « *before any other statements are executed* » (note 2, §2.1).

1. **Mauvais répertoire.** « When R is started at later time **from the same directory** it reloads the workspace from this file. » Le `.RData` est lié au dossier de lancement.
2. **Restauration désactivée.** `--no-restore` , `--no-restore-data` ou `--vanilla` au démarrage.
3. **Objets cachés.** Si les objets recherchés ont un nom **commençant par un point**, `ls()` ne les montre pas (*R Language Definition* §10.3.2) — utiliser `ls(all.names = TRUE)` .

Et dans les trois cas, **aucune erreur n'est levée** : l'absence de `.RData` est le cas normal d'une première session.

```

help("[[")      # ou ?"["
help("for")     # mot a sens syntaxique : les guillemets sont OBLIGATOIRES
example(solve)  # execute les exemples de la page d'aide

```

*« For a feature specified by **special characters**, the argument must be enclosed in double or single quotes, making it a "character string". This is also necessary for **a few words with syntactic meaning including if , for and function** . »*

Ordre	Étape	Coupée par
1	Renviron de site (R_ENVIRON , sinon R_HOME/etc/Renviron.site) puis utilisateur (R_ENVIRON_USER , sinon .Renviron courant puis personnel)	--no-environ
2	profil de site (R_PROFILE , sinon R_HOME/etc/Rprofile.site)	--no-site-file
3	profil utilisateur (R_PROFILE_USER , sinon .Rprofile courant puis personnel)	--no-init-file
4	.RData du répertoire courant	--no-restore , - -no-restore-data
5	.First() , si elle existe	—

.First() peut résider dans **.RData** — donc l'étape 4 peut **fournir** l'étape 5. Fin de session : **.Last()** .

Parce qu'il **coupe la séquence entière d'un seul coup** : « Combine **--no-save** , **--no-environ** , **--no-site-file** , **--no-init-file** and **--no-restore** . Under Windows, this also includes **--no-Rconsole** . »

Le test est **discriminant** : le problème disparaît → il vient d'un fichier de démarrage, qu'on remonte alors dans l'ordre ; il persiste → il vient de R ou d'un paquet, et les fichiers de démarrage sont hors de cause.

*« Note that **R CMD** **does not of itself use any R startup files** (in particular, **neither user nor site Renviron files**), and **all of the R processes run by these tools (except BATCH) use --no-restore .** »*

Votre session lit `.Renviron` et `.Rprofile` ; `R CMD check` **ne les lit pas**. Tout ce qui dépend d'un `R_LIBS` personnalisé, d'un `library()` posé dans `.Rprofile` ou d'un objet venant de `.RData` **n'existe pas** dans le processus de vérification. C'est le mécanisme exact du « ça marche chez moi ».

```
#!/usr/bin/env Rscript
args <- commandArgs(TRUE)
if (length(args) != 2L) q(status = 1)
# ... traitement ...
q(status = 0)
```

Puis `chmod 755 runfoo`, et `runfoo arg1 arg2`.

« At least in Bourne and bash shells, the `#!` mechanism **does not allow extra arguments** like `#!/usr/bin/env Rscript --vanilla .` »

En R CMD BATCH, la même chose s'écrit `R CMD BATCH "--args arg1 arg2" foo.R &` — les arguments passent par `--args`, dont le rôle est précisément de « *cause the rest of the command line to be skipped* ».

« `stdin()` refers to **the script file** to allow such traditional usage » — c'est-à-dire l'idiome `chem <- scan(n = 24)` suivi des données **écrites dans le script lui-même**.

*« If you want to refer to **the process's** `stdin` , use `"stdin"` as a file connection, e.g. `scan("stdin", ...)` . »*

Et l'inverse en *here document* (`R --no-echo <<EOF`) : « *here* `stdin()` refers to **the program source** and `"stdin"` **will not be usable** ». Les deux formes ont donc des sémantiques d'entrée opposées.

`x<-5` **assigne 5 à x** . `x < -5` **teste si x est inférieur à -5**.

R Language Definition §10.3.5 : « *stretches of whitespace characters serve to **delimit tokens in case of ambiguity**, (compare `x<-5` and `x < -5`)* ». L'espace **est** significatif ici, non pas en général, mais parce que `<-` et `<` - sont deux découpages en jetons du même texte, et que **le parseur choisit le plus long**.

Le remède : mettre des espaces autour de `<-` **toujours**.

*« The first two are often abbreviated as `T` and `F` , respectively. Note however that **`T` and `F` are just variables which are set to `TRUE` and `FALSE` by default, but are not reserved words and hence can be overwritten by the user**. Hence, **you should always use `TRUE` and `FALSE`** . »* (§2.4)

La preuve par la liste des mots réservés (*R Language Definition §10.3.3*) : `TRUE` et `FALSE` y figurent, **`T` et `F` non**. Un `T <- 0` quelque part dans un script rend tout `if (x == T)` silencieusement faux.

`Expt` contient les entiers 1 à 5, mais ce sont des **étiquettes d'expérience**, pas des quantités. Sans `factor()` , tout modèle traiterait la différence entre l'expérience 4 et l'expérience 2 comme une différence **numérique de 2 unités** — ce qui n'a aucun sens.

Et `plot` **change de comportement** : parce que son premier argument **est un facteur**, il produit des **boîtes à moustaches** (« *Compare the five experiments with simple boxplots* ») au

lieu d'un nuage de points. C'est le dispatch S3 : la fonction générique choisit sa méthode **d'après la classe** de l'argument.

Environ 4 095 octets — « *not characters* » (§1.8). Elle reste en vigueur **même avec redirection** `<` `>` (annexe B.1).

Le danger n'est pas la limite mais **l'absence d'uniformité** (note 3) : « *some of the consoles will not allow you to enter more, and amongst those which do **some will silently discard the excess** and some will use it as **the start of the next line*** ». Trois comportements : refus, **troncature silencieuse**, ou report. Le second est le pire — le calcul se fait sur une commande mutilée sans qu'aucune erreur ne le signale.

Le remède : `source()` un fichier, dont les commandes sont « *of unlimited length* » (note 4, §1.10).

```
oldpar <- par(no.readonly = TRUE)  # sauver
par(pty = "s")                    # modifier
# ... traces ...
par(oldpar)                        # restaurer
```

« *Save the plotting parameters and set the plotting region to "square"* » puis « *...and restore the old graphics parameters* ».

`par()` **est global et persistant** : un `par(pty = "s")` non annulé rend **carrées toutes les figures suivantes de la session**. L'argument `no.readonly = TRUE` sélectionne les paramètres **modifiables**, seuls repassables à `par()`.

Flashcards

Question	Réponse
R en une phrase ?	Une suite intégrée pour manipulation de données, calcul et graphiques
Pourquoi « environnement » ?	Système planifié et cohérent , pas une accrétion d'outils inflexibles
Le langage de R ?	S , développé aux Bell Labs (Becker, Chambers, Wilks)
Le « Blue Book » ?	<i>The New S Language</i> — la référence de base pour R
Le « White Book » ?	<i>Statistical Models in S</i> — les modèles
Le « Green Book » ?	<i>Programming with Data</i> — les classes formelles (S4)
Différence R / SAS-SPSS ?	R donne peu de sortie et stocke le résultat dans un objet
Combien de paquets livrés avec R ?	Environ 25 (standard + recommended)
L'invite par défaut ?	<code>></code> ; continuation <code>+</code>
Comment quitter ?	<code>q()</code>
Les trois réponses à la question de sortie ?	<code>yes</code> (sauver et quitter) · <code>no</code> (quitter) · <code>cancel</code> (revenir dans la session)
Qu'est-ce que l'espace de travail ?	La collection d'objets actuellement stockés
Lister les objets ?	<code>ls()</code> ou <code>objects()</code>
Les effacer ?	<code>rm(x, y, ...)</code>
Ce que <code>ls()</code> ne montre pas ?	Les noms commençant par un point
Les deux fichiers écrits à la sortie ?	<code>.RData</code> (objets) et <code>.Rhistory</code> (commandes)
Quand sont-ils relus ?	Au démarrage depuis le même répertoire
Pourquoi sont-ils invisibles ?	Le point initial , sur UNIX, macOS et Windows
Pourquoi un dossier par analyse ?	Pour que des noms comme <code>x</code> et <code>y</code> restent utilisables
Les deux sortes de commandes ?	Expression et assignation

Question	Réponse
Une expression ?	Évaluée, affichée , valeur perdue
Une assignation ?	Évaluée, rangée , rien d'affiché
Récupérer la dernière valeur ?	<code>.Last.value</code> , avant toute autre instruction
R est-il sensible à la casse ?	Oui — <code>A</code> et <code>a</code> sont deux symboles
Caractères permis dans un nom ?	Alphanumériques + <code>.</code> + <code>_</code>
Premier caractère d'un nom ?	Une lettre ou un point (non suivi d'un chiffre)
Noms pour du code portable ?	A–Z , a–z , 0–9 seulement
Combien de mots réservés ?	<code>if else repeat while function for in next break</code> + <code>TRUE FALSE NULL</code> <code>Inf NaN</code> + les cinq <code>NA...</code> + <code>...</code> <code>..1</code> <code>..2</code>
<code>T</code> et <code>F</code> sont-ils réservés ?	Non — ce sont des variables réassignables
Séparateurs de commandes ?	<code>;</code> ou saut de ligne
Groupement ?	Les accolades <code>{ }</code>
Commentaire ?	<code>#</code> jusqu'à la fin de la ligne
Où <code>#</code> est-il interdit ?	Dans une chaîne et dans la liste d'arguments d'une définition de fonction
Limite d'une ligne de console ?	Environ 4 095 octets
Les trois comportements au-delà ?	Refus · troncature silencieuse · report sur la ligne suivante
<code>x<-5</code> contre <code>x < -5</code> ?	Assignation contre comparaison à –5
Où mettre <code>else</code> à la console ?	Sur la même ligne que <code>}</code>
Aide sur un nom ?	<code>?solve</code> ou <code>help(solve)</code>
Aide sur <code>[]</code> ?	<code>help("[")</code> — guillemets obligatoires
Aide sur <code>if</code> ?	<code>help("if")</code> — mot à sens syntaxique
Chercher dans l'aide ?	<code>??</code> ou <code>help.search()</code>

Question	Réponse
Faire tourner les exemples ?	<code>example(topic)</code>
Aide en HTML ?	<code>help.start()</code>
Le lien le plus utile de <code>help.start()</code> ?	Search Engine and Keywords
Effet de <code>help.start()</code> sous UNIX ?	Les requêtes suivantes partent dans le navigateur
Exécuter un fichier de commandes ?	<code>source("commands.R")</code>
Détourner la sortie ?	<code>sink("record.lis")</code> ; <code>sink()</code> pour revenir
Longueur des commandes dans un fichier ?	Illimitée
Étape 1 du démarrage ?	Les fichiers Renviron (site puis utilisateur)
Étape 2 ?	<code>Rprofile.site</code>
Étape 3 ?	<code>.Rprofile</code> (courant puis personnel)
Étape 4 ?	<code>.RData</code>
Étape 5 ?	<code>.First()</code>
Fin de session ?	<code>.Last()</code>
Où peut résider <code>.First()</code> ?	Dans un profil ou dans <code>.RData</code>
Trois variables de Renviron ?	<code>R_PAPERSIZE</code> , <code>R_PRINTCMD</code> , <code>R_LIBS</code>
Contrainte sur <code>TMPDIR</code> ?	Non défini , ou pointant vers un endroit valide
Que combine <code>--vanilla</code> ?	<code>--no-save</code> <code>--no-environ</code> <code>--no-site-file</code> <code>--no-init-file</code> <code>--no-restore</code> (+ <code>--no-Rconsole</code>)
Sous Windows, quel exécutable en ligne de commande ?	<code>Rterm.exe</code> , préféré à <code>R</code>
Que fait <code>RHOME</code> ?	Affiche le répertoire d'installation de R
Défaut de restauration de <code>.RData</code> ?	Restaurer
<code>-f</code> et <code>-e</code> impliquent quoi ?	<code>--no-save</code>

Question	Réponse
Limite des expressions <code>-e</code> ?	10 000 octets au total
Peut-on mêler <code>-e</code> et <code>-f</code> ?	Non
Que fait <code>--no-echo</code> / <code>-s</code> ?	R aussi silencieux que possible ; implique <code>--quiet</code> et <code>--no-save</code>
Que fait <code>--args</code> ?	Fait ignorer le reste de la ligne , à lire par <code>commandArgs(TRUE)</code>
Défauts de déclenchement du GC ?	6 Mo (vecteurs) et 350 k (cellules cons)
Défaut et maximum de <code>--max-ppsize</code> ?	10 000 , maximum 100 000
Où vont avertissements et erreurs ?	Sur <code>stderr</code>
Que lance <code>R CMD BATCH</code> ?	<code>R --restore --save</code>
Deux outils <code>R CMD</code> pour les paquets ?	<code>build</code> et <code>check</code> (+ <code>INSTALL</code> , <code>REMOVE</code>)
<code>R CMD</code> lit-il vos fichiers de démarrage ?	Non — et impose <code>--no-restore</code> sauf pour <code>BATCH</code>
La façade simple pour les scripts ?	<code>Rscript</code>
Récupérer les arguments d'un script ?	<code>commandArgs(TRUE)</code> → vecteur de caractères
Shebang portable ?	<code>#!/usr/bin/env Rscript</code>
Peut-on y ajouter <code>--vanilla</code> ?	Non en Bourne/bash
Rendre le script exécutable ?	<code>chmod 755</code>
Renvoyer un code de sortie ?	<code>q(status = ...)</code>
À quoi renvoie <code>stdin()</code> dans un script ?	Au fichier de script lui-même
Et l'entrée du processus ?	La connexion <code>"stdin"</code> , ex. <code>scan("stdin", ...)</code>
Et en <i>here document</i> ?	<code>stdin()</code> = la source du programme ; <code>"stdin"</code> inutilisable
<code>Rscript</code> accepte-t-il un script vide ?	Non

Question	Réponse
Quelle bibliothèque pour l'édition de ligne ?	GNU readline
Savoir laquelle on a ?	<code>extSoftVersion()</code>
La désactiver ?	<code>--no-readline</code>
Chercher dans l'historique ?	<code>C-r texte</code> (annuler par <code>C-g</code>)
Début / fin de ligne ?	<code>C-a</code> / <code>C-e</code>
Reculer d'un mot ?	<code>M-b</code> (ou <code>ESC b</code>)
Couper jusqu'à la fin ?	<code>C-k</code> ; coller : <code>C-y</code>
Transposer deux caractères ?	<code>C-t</code>
La casse compte-t-elle pour Meta ?	Oui
Personnaliser les touches ?	<code>~/.inputrc</code> , section <code>\$if R ... \$endif</code>
Idiome de sauvegarde des paramètres graphiques ?	<code>oldpar <- par(no.readonly = TRUE) ... par(oldpar)</code>
Pourquoi <code>factor(mm\$Expt)</code> dans l'annexe A ?	<code>Expt</code> est une étiquette , pas une quantité
Que devient <code>plot(Expt, Speed)</code> alors ?	Des boîtes à moustaches — dispatch S3
Que fait <code>update(fm, . ~ . - Run)</code> ?	Le même modèle sans <code>Run</code>
Que vaut <code>rnorm(x)</code> ?	Autant de tirages que <code>x</code> a d'éléments
Pourquoi <code>sqrt(runif(100))</code> pour le disque ?	L'aire croît en r^2 — sans la racine, les points s'entassent au centre